



Aula 3 – Git Flow na prática

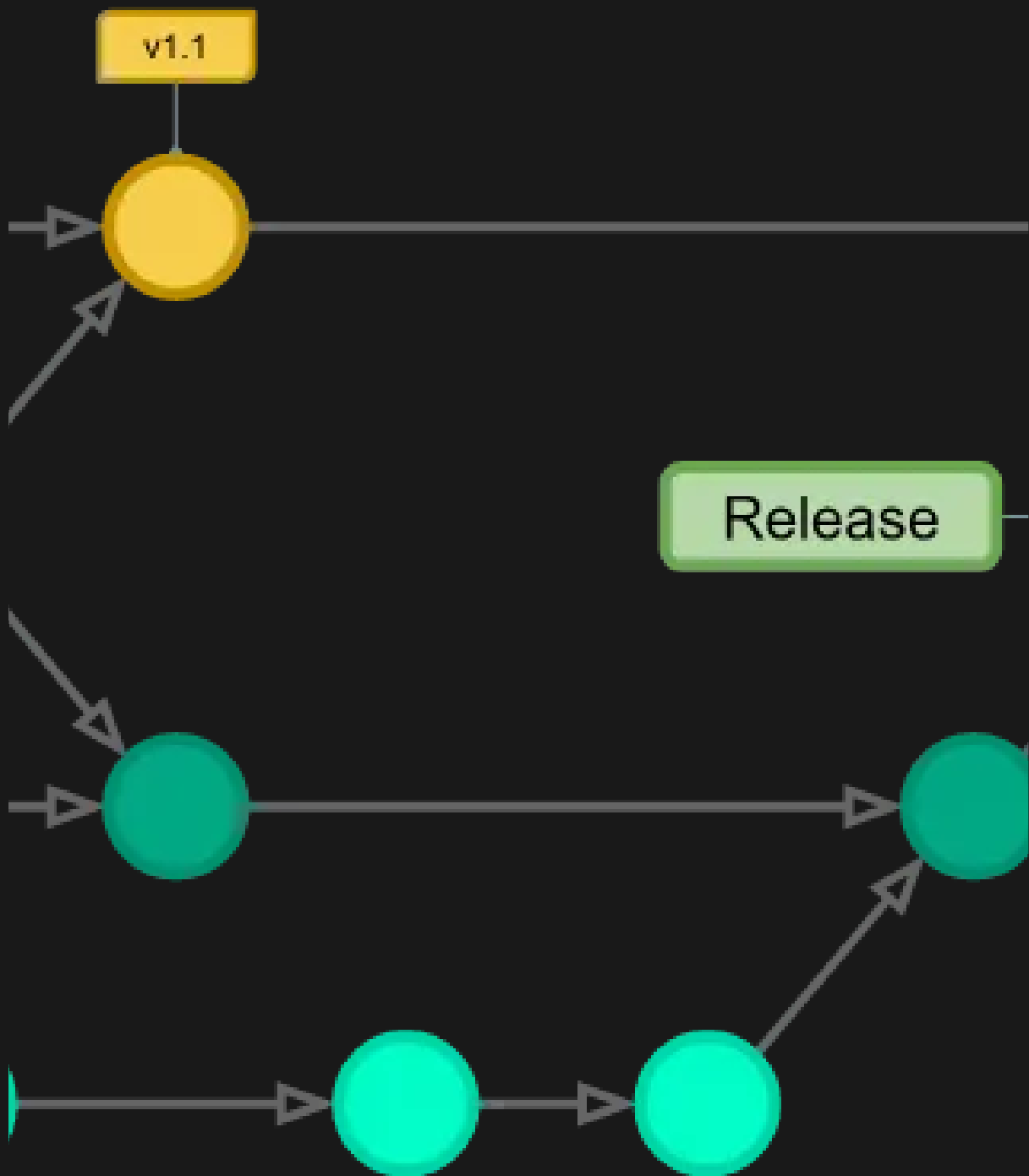
Objetivos

- Compreender o que é o **Git Flow** e por que usá-lo em times/projetos organizados
 - Criar branches com propósito claro: `main`, `develop`, `feature`, `hotfix`, `release`
 - Simular uma rotina de trabalho em equipe usando Git Flow no projeto **TrackFit**
 - Aplicar boas práticas de mensagens de commit
-

O que é Git Flow?

- **Modelo de ramificação (branching model)** que organiza o fluxo de trabalho com Git em ambientes de desenvolvimento real
 - Ajuda a manter separação entre **código estável**, **funcionalidades em desenvolvimento**, e **correções urgentes**
 - Muito usado em projetos com múltiplos desenvolvedores
-

Estrutura do Git Flow



<u>Branch</u>	<u>Responsabilidade</u>
main/master	código em produção
develop	base para desenvolvimento
feature/*	funcionalidades novas
release/*	preparação para release
hotfix/*	correções urgentes direto da produção

Simulação no projeto TrackFit

1. Iniciar repositório local (se ainda não tiver feito)

```
git init git remote add origin <url-do-repo> git checkout -b main
```

2. Criar branch `develop`

```
git checkout -b develop git push origin develop
```

Criando uma branch de funcionalidade

Exemplo: criar tela de cadastro

```
git checkout -b feature/formulario-cadastro
```

Padrão: `feature/<nome-da-feature>`

Seja descritivo e use `-` para separar palavras

Boas práticas de commit

```
git add . git commit -m "feat: cria componente de formulário de treino"
```

Prefixos úteis:

- `feat:` nova funcionalidade
- `fix:` correção de bug
- `refactor:` refatoração de código
- `style:` ajuste de layout ou formatação
- `docs:` alterações na documentação
- `test:` criação ou melhoria de testes

<https://www.conventionalcommits.org/en/v1.0.0/>

Após concluir a funcionalidade

```
git checkout develop git merge feature/formulario-cadastro git push origin develop
```

Corrigindo algo direto da main (hotfix)

```
git checkout main git pull origin main git checkout -b hotfix/fix-header-logo
```

Após correção:

```
git commit -m "fix: corrige tamanho da logo no header" git checkout main git merge hotfix/fix-header-logo git checkout develop git merge hotfix/fix-header-logo git push origin main git push origin develop
```

Checklist da Aula

- ☐ Entendeu a estrutura do Git Flow
- ☐ Criou as branches `develop` e `feature`
- ☐ Fez commits com boas mensagens
- ☐ Fez merge da `feature` na `develop`
- ☐ Corrigiu um bug simulado com `hotfix`

Atividade prática

1. Criar uma branch `feature/header`.
2. Criar um componente `Header.tsx` com um título.
3. Fazer commit com a mensagem: `feat: cria header principal da aplicação`.
4. Mergiar na branch `develop`.
5. Criar uma branch `hotfix/logo`, alterar algo no título e aplicar a lógica do hotfix.

